

Uso de IA Generativa:

En este práctico **NO se permite** el uso de estas herramientas.

Práctico 4 - Sincronización por grupos y bibliotecas de primitivas generales

El objetivo de este práctico es poner en práctica conceptos relacionados con el uso de Cooperative Groups y bibliotecas de alto desempeño, en particular Thrust y CUB. Los tiempos de ejecución deben tomarse como el promedio de 10 ejecuciones (presentando únicamente el promedio y desviación estándar en el informe). Para obtener este dato fácilmente es conveniente utilizar la herramienta NsightSystems de la siguiente forma:

```
sbatch lanzar.sh nsys profile --stats true ./programa
```

Ejercicio 1

Implementación de Exclusive Scan

La operación de *scan* es una primitiva fundamental en el cómputo paralelo. Dado un vector de entrada x y un operador binario asociativo, el *scan* calcula los resultados parciales acumulados al aplicar dicho operador sobre el vector.

En este ejercicio se trabajará con la suma exclusiva. Para un arreglo x de entrada y un arreglo y de salida, $y_0 = 0$ y $y_i = \sum_{k=0}^{i-1} x_k$. Por ejemplo,

$x = [3, 1, 4, 2, 5]$

debe producir:

$y = [0, 3, 4, 8, 10]$

1. Implemente una rutina en CUDA que calcule la suma exclusiva de un vector de longitud arbitraria. La rutina debe ser razonablemente eficiente en el acceso a la memoria y puede utilizar más de un *kernel*.
2. Resuelva el mismo problema utilizando CUB y Thrust.
3. Compare las tres implementaciones considerando el tiempo de ejecución y la complejidad de cada una. Evalúe la implementación con vectores de largo: $N = 1024 \times 2^k$ para al menos $k = 1, 2, \dots, 10$.

Nota:

No es necesario resolver el problema con un único *kernel*. Una estrategia posible es calcular primero un *scan* parcial por bloque, guardar la suma total de cada bloque en un vector auxiliar, aplicar un *scan* sobre ese vector y luego sumar el desplazamiento correspondiente a cada bloque.

Ejercicio 2

Uso de Cooperative Groups

El objetivo de este ejercicio es utilizar Cooperative Groups para expresar reducciones sobre conjuntos de hilos dentro de un bloque de CUDA. Se trabajará con un vector de entrada x de largo N , almacenado en memoria global. En todos los casos se utilizará la suma como operación de reducción. El resultado será un vector y de tamaño `cant_segmentos`, donde $y[i]$ contendrá el resultado de la reducción del segmento i .

1. Implemente un *kernel* en CUDA que reduzca el vector dividiéndolo en segmentos regulares de 8 elementos. Para esto, utilice `cooperative_groups::tiled_partition<8>`.

Por ejemplo:

$x = [3, 1, 4, 2, 5, 7, 1, 6, 8, 2, 9, 1, 3, 4, 5, 2]$

debe producir:

```
y = [29, 34]
```

2. Considere ahora segmentos de largo variable. Para determinar a qué segmento pertenece cada elemento del vector, se recibirá un vector de etiquetas **e**, tal que en **e[i]** se encuentra el índice de segmento que le corresponde al elemento **x[i]**. Para esta parte, utilice `cooperative_groups::labeled_partition`. Por ejemplo,

```
x = [3, 1, 4, 2, 5, 7, 1, 6]
e = [0, 0, 0, 1, 1, 1, 2, 2]
```

debe producir,

```
y = [8, 14, 7].
```

A efectos de simplificar el problema, se puede asumir que:

- Las apariciones de una misma etiqueta forman un único segmento contiguo. Es decir, si $e[i] = e[j]$ con $i < j$, entonces $e[k] = e[i]$ para todo $i \leq k \leq j$.
- Ningún segmento cruza una frontera que sea un múltiplo de 32. Es decir, para todo entero $b > 0$ tal que $32b < N$ se cumple que: $e[32b - 1] \neq e[32b]$.

Ejercicio 3	Uso de “building blocks” para paralelizar rutinas complejas
--------------------	---

En muchas aplicaciones paralelas, es necesario reorganizar los datos antes de una etapa de cómputo posterior. Considere un arreglo de enteros no negativos **input** de tamaño N . Cada elemento pertenece a un *bin* determinado por $bin = input[i]/K$, donde K es un entero positivo. En este práctico se define $K = 10$.

Se desea construir tres arreglos:

- Un arreglo **output** con los elementos agrupados por bin, preservando el orden relativo dentro de cada bin.
- Un arreglo **bin_counts** que contiene la cantidad de elementos en cada bin.
- Un arreglo **bin_offsets** que contiene la posición inicial de cada bin en **output**.

Por ejemplo, si

```
input = [3, 17, 8, 21, 5, 14, 30, 9]
K = 10
```

los bins correspondientes son

```
0 1 0 2 0 1 3 0
```

y la salida esperada sería

```
output      = [3, 8, 5, 9, 17, 14, 21, 30]
bin_counts  = [4, 2, 1, 1]
bin_offsets = [0, 4, 6, 7]
```

Se pide

1. Implementar una rutina paralela, utilizando primitivas de Thrust, que genere los arreglos `output`, `bin_counts` y `bin_offsets` a partir del agrupamiento obtenido.
2. Comparar la implementación desarrollada con una versión secuencial proporcionada en el material.

Entrega

- Se debe entregar un informe en PDF con la solución de los ejercicios que contenga, **como máximo, 5 páginas** (sin contar índices, carátulas, figuras, etc.).
- Entregar un archivo comprimido con el código fuente y los scripts necesarios para ejecutar un caso de prueba de manera sencilla.